

Elliptic Bitcoin Fraud Network Analysis

HNG Stage 7 - Blockchain Data Analytics

The goal of this project is to analyze the Elliptic Bitcoin transaction dataset and uncover patterns that point to illicit financial activity. The dataset comes in three files, transaction features, a directed edge list, and class labels. I'll work through data preparation, network construction, fraud pattern detection, and finally build an interactive dashboard to present the findings.

Task 1: Data Preparation & Validation

Before doing any analysis I need to understand what the raw data actually looks like. I'll load each file separately, inspect the structure, handle any issues, then merge everything into one clean dataframe.

```
In [1]: import pandas as pd
import numpy as np
```

Load the three raw files

I'm loading them separately on purpose, merging without knowing what each file contains is a good way to introduce silent errors that are hard to trace later.

```
In [2]: features_df = pd.read_csv(r"C:\Users\hp\Downloads\archive (2)\elliptic_bitcoin_data
edges_df = pd.read_csv(r"C:\Users\hp\Downloads\archive (2)\elliptic_bitcoin_data
classes_df = pd.read_csv(r"C:\Users\hp\Downloads\archive (2)\elliptic_bitcoin_data

print(f"Features : {features_df.shape}")
print(f"Edges : {edges_df.shape}")
print(f"Classes : {classes_df.shape}")
```

```
Features : (203769, 167)
Edges : (234355, 2)
Classes : (203769, 2)
```

Inspect the features file before naming columns

The features file has no headers so I need to look at the raw data first to understand what each column actually represents before I start renaming anything.

```
In [3]: print("First 3 rows, first 5 columns:")
print(features_df.iloc[:, :5].head(3))

print("\nColumn 0 sample values:")
```

```
print(features_df.iloc[:, 0].head(10).tolist())

print("\nColumn 1 unique values:")
print(features_df.iloc[:, 1].unique())
```

First 3 rows, first 5 columns:

	0	1	2	3	4
0	230425980	1	-0.171469	-0.184668	-1.201369
1	5530458	1	-0.171484	-0.184668	-1.201369
2	232022460	1	-0.172107	-0.184668	-1.201369

Column 0 sample values:

[230425980, 5530458, 232022460, 232438397, 230460314, 230459870, 230333930, 230595899, 232013274, 232029206]

Column 1 unique values:

[1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24
25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48
49]

From the output above:

- Column 0 has large unique integers - these are transaction IDs
- Column 1 has values 1 to 49 these are time steps (the dataset covers 49 time periods)
- Everything from column 2 onward are decimal numbers - the anonymized transaction features

This matches the Elliptic dataset documentation on Kaggle.

Assign column names

Keeping integer column names causes problems in pandas, merges behave differently and if i save and reload the file the integers become strings and break the code. I'll rename everything to proper string names.

The feature names like `feat_1`, `feat_2` are not meaningful since all 165 features are anonymized, but at least they're consistent string labels.

```
In [4]: # Use shape[1] so this works regardless of how many columns the file has
num_features = features_df.shape[1] - 2
feature_cols = ['txId', 'time_step'] + [f'feat_{i}' for i in range(1, num_features)]
features_df.columns = feature_cols

print(f"Columns assigned: {len(feature_cols)} total")
print(f"Matches file width: {features_df.shape[1]}")
features_df.head()
```

Columns assigned: 167 total

Matches file width: 167

Out[4]:

	txld	time_step	feat_1	feat_2	feat_3	feat_4	feat_5	feat_6
0	230425980	1	-0.171469	-0.184668	-1.201369	-0.121970	-0.043875	-0.113002
1	5530458	1	-0.171484	-0.184668	-1.201369	-0.121970	-0.043875	-0.113002
2	232022460	1	-0.172107	-0.184668	-1.201369	-0.121970	-0.043875	-0.113002
3	232438397	1	0.163054	1.963790	-0.646376	12.409294	-0.063725	9.782742
4	230460314	1	1.011523	-0.081127	-1.201369	1.153668	0.333276	1.312656

5 rows × 167 columns



Standardize column names across all three files

All three files need to use the same column name for transaction IDs so the merge works cleanly.

```
In [5]: print("Edges columns :", edges_df.columns.tolist())
        print("Classes columns:", classes_df.columns.tolist())
```

```
edges_df.columns = ['txId1', 'txId2']
classes_df.columns = ['txId', 'class']
```

```
Edges columns : ['txId1', 'txId2']
```

```
Classes columns: ['txId', 'class']
```

Check for duplicates

Duplicate transaction records would inflate node importance scores later. If a transaction appears twice it would look like it has twice the connections it actually has.

```
In [6]: dup_features = features_df.duplicated(subset='txId').sum()
        dup_classes = classes_df.duplicated(subset='txId').sum()
        dup_edges = edges_df.duplicated().sum()

        print(f"Duplicate IDs in features : {dup_features}")
        print(f"Duplicate IDs in classes : {dup_classes}")
        print(f"Duplicate rows in edges : {dup_edges}")

        # Drop any duplicates if found – keeping first occurrence
        features_df.drop_duplicates(subset='txId', keep='first', inplace=True)
        classes_df.drop_duplicates(subset='txId', keep='first', inplace=True)
        edges_df.drop_duplicates(inplace=True)
```

```
Duplicate IDs in features : 0
```

```
Duplicate IDs in classes : 0
```

```
Duplicate rows in edges : 0
```

Check for missing values

Missing feature values would silently distort centrality calculations. I already checked this using `.info()` but I'll verify programmatically too.

```
In [7]: missing_total = features_df.isnull().sum().sum()
print(f"Total missing values in features file: {missing_total}")
```

Total missing values in features file: 0

Handle the class labels

The classes file uses `'1'` for illicit, `'2'` for licit, and `'unknown'` for unlabeled transactions. I'll keep all three groups, dropping the unknown transactions would remove about 75% of the dataset and break the graph structure since those nodes still appear in the edge list.

I'll also create a numeric version of the label for use in scoring later.

```
In [8]: print("Raw class distribution:")
print(classes_df['class'].value_counts())

# Normalize - some versions of the file have NaN instead of 'unknown'
classes_df['class'] = classes_df['class'].fillna('unknown')

# Numeric label for scoring: 1 = illicit, 0 = licit, -1 = unknown
label_map = {'1': 1, '2': 0, 1: 1, 2: 0, 'unknown': -1}
classes_df['label'] = classes_df['class'].map(label_map)

print("\nCleaned class distribution:")
print(classes_df['class'].value_counts())
```

Raw class distribution:

```
class
unknown    157205
2           42019
1           4545
Name: count, dtype: int64
```

Cleaned class distribution:

```
class
unknown    157205
2           42019
1           4545
Name: count, dtype: int64
```

Merge into one master dataframe

I'm using a left join from features onto classes so all transactions are kept regardless of whether they have a label. An inner join would silently drop unlabeled transactions and

break the network later.

```
In [9]: merged_df = features_df.merge(classes_df, on='txId', how='left')

# Transactions that didn't appear in the classes file at all
# get filled as unknown
merged_df['class'] = merged_df['class'].fillna('unknown')
merged_df['label'] = merged_df['label'].fillna(-1).astype(int)

print(f"Merged dataframe shape: {merged_df.shape}")
merged_df.head()
```

Merged dataframe shape: (203769, 169)

```
Out[9]:
```

	txId	time_step	feat_1	feat_2	feat_3	feat_4	feat_5	feat_6
0	230425980	1	-0.171469	-0.184668	-1.201369	-0.121970	-0.043875	-0.113002
1	5530458	1	-0.171484	-0.184668	-1.201369	-0.121970	-0.043875	-0.113002
2	232022460	1	-0.172107	-0.184668	-1.201369	-0.121970	-0.043875	-0.113002
3	232438397	1	0.163054	1.963790	-0.646376	12.409294	-0.063725	9.782742
4	230460314	1	1.011523	-0.081127	-1.201369	1.153668	0.333276	1.312656

5 rows × 169 columns



Validate edge list integrity

Some transactions referenced in the edge list might not have feature records. These are called ghost nodes. I'll check how many there are and decide what to do with them.

```
In [10]: known_ids = set(merged_df['txId'])

ghost_sources = (~edges_df['txId1'].isin(known_ids)).sum()
ghost_targets = (~edges_df['txId2'].isin(known_ids)).sum()

print(f"Edge sources not in features: {ghost_sources}")
print(f"Edge targets not in features: {ghost_targets}")

# Keeping all edges – removing them could disconnect real fraud clusters.
# Ghost nodes will be treated as unknown during analysis.
print("Decision: keeping all edges. Ghost nodes labeled as unknown.")
```

Edge sources not in features: 0

Edge targets not in features: 0

Decision: keeping all edges. Ghost nodes labeled as unknown.

Final summary before export

```
In [11]: print(f"Total transactions : {len(merged_df):,}")
print(f"Total edges          : {len(edges_df):,}")
print(f"Time steps          : {merged_df['time_step'].nunique()}")
print(f"Illicit             : {(merged_df['label'] == 1).sum():,}")
print(f"Licit                : {(merged_df['label'] == 0).sum():,}")
print(f"Unlabeled            : {(merged_df['label'] == -1).sum():,}")
print(f"Missing values       : {merged_df.isnull().sum().sum():,}")
```

```
Total transactions : 203,769
Total edges        : 234,355
Time steps         : 49
Illicit            : 4,545
Licit              : 42,019
Unlabeled          : 157,205
Missing values     : 0
```

Export cleaned files

```
In [12]: print("Checking before export:")
print(f"  Shape          : {merged_df.shape}")
print(f"  Illicit          : {(merged_df['label'] == 1).sum():,}")
print(f"  Licit            : {(merged_df['label'] == 0).sum():,}")
print(f"  Unlabeled        : {(merged_df['label'] == -1).sum():,}")
print(f"  Missing values   : {merged_df.isnull().sum().sum():,}")
print(f"  Edge rows        : {len(edges_df):,}")

merged_df.to_csv('elliptic_cleaned_merged.csv', index=False)
edges_df.to_csv('elliptic_edges_cleaned.csv', index=False)

print("\nSaved:")
print("  elliptic_cleaned_merged.csv")
print("  elliptic_edges_cleaned.csv")
```

```
Checking before export:
  Shape          : (203769, 169)
  Illicit         : 4,545
  Licit           : 42,019
  Unlabeled       : 157,205
  Missing values  : 0
  Edge rows       : 234,355
```

```
Saved:
  elliptic_cleaned_merged.csv
  elliptic_edges_cleaned.csv
```

Task 2: Network Construction & Subgraph Selection

Now I'll model the transactions as a directed graph where each node is a transaction and each edge is a payment flow. I won't analyze the full 200k+ node graph. computing centrality on that would take hours. Instead I'll use a structured approach to extract a meaningful subgraph centered around the most important transactions.

```
In [13]: import networkx as nx
import random
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from pyvis.network import Network
```

Build the full directed graph

I'm using a directed graph (DiGraph) because payment flows have direction - money moves FROM one transaction TO another. An undirected graph would lose that and make in/out degree analysis meaningless.

```
In [14]: merged_df = pd.read_csv('elliptic_cleaned_merged.csv')
edges_df = pd.read_csv('elliptic_edges_cleaned.csv')

G = nx.from_pandas_edgelist(
    edges_df,
    source='txId1',
    target='txId2',
    create_using=nx.DiGraph()
)

print(f"Nodes: {G.number_of_nodes():,}")
print(f"Edges: {G.number_of_edges():,}")
```

Nodes: 203,769

Edges: 234,355

Attach labels to each node

By default networkx nodes are just IDs with no attributes. I need to tag each node with its class label and time step so I can use them during visualization and fraud detection.

```
In [15]: label_lookup = merged_df.set_index('txId')['class'].to_dict()
timestep_lookup = merged_df.set_index('txId')['time_step'].to_dict()

nx.set_node_attributes(G, label_lookup, name='class')
nx.set_node_attributes(G, timestep_lookup, name='time_step')

# Ghost nodes (in edge list but not in features) get Labeled unknown
for node in G.nodes():
    if G.nodes[node].get('class') is None:
        G.nodes[node]['class'] = 'unknown'
        G.nodes[node]['time_step'] = -1
```

```
print("Node attributes attached.")
print(f"Sample: {dict(list(G.nodes(data=True))[:1])}")
```

Node attributes attached.

Sample: {230425980: {'class': 'unknown', 'time_step': 1}}

Sample 20,000 nodes for betweenness centrality

Betweenness centrality tells me how often a node sits on the shortest path between two other nodes. High betweenness means a transaction is acting as a bridge or relay, exactly the kind of behavior associated with money laundering.

Computing this on all 200k+ nodes would take hours so I'll sample 20,000 as instructed. Setting a random seed means the sample is the same every time I run the notebook.

```
In [16]: random.seed(42)
all_nodes = list(G.nodes())
sample_nodes = random.sample(all_nodes, 20000)
H = G.subgraph(sample_nodes)

print(f"Sample subgraph: {H.number_of_nodes():,} nodes, {H.number_of_edges():,} edges")
```

Sample subgraph: 20,000 nodes, 2,381 edges

Compute betweenness centrality and select top 50 nodes

```
In [17]: print("Computing betweenness centrality...")

try:
    bc = nx.betweenness centrality(H, normalized=True)
    print("Exact betweenness computed.")
except Exception as e:
    print(f"Falling back to approximation: {e}")
    bc = nx.betweenness centrality(H, k=500, normalized=True)

top50 = sorted(bc, key=bc.get, reverse=True)[:50]

print(f"Top 50 selected. Highest score: {max(bc.values()):.6f}")
print(f"50th node score: {bc[top50[49]]:.6f}")
```

Computing betweenness centrality...

Exact betweenness computed.

Top 50 selected. Highest score: 0.000000

50th node score: 0.000000

Extract ego network of the top 50 nodes

The ego network of a node means that node plus all its direct neighbors. I take the union across all 50 top nodes. This gives me a subgraph that's anchored around the most structurally important transactions, not just a random slice of the data.

The task requires at least 5,000 nodes. If the 1-hop network falls short I'll keep expanding outward until it does.

```
In [18]: ego_nodes = set(top50)

for node in top50:
    ego_nodes.update(G.predecessors(node))
    ego_nodes.update(G.successors(node))

subgraph = G.subgraph(ego_nodes).copy()

print(f"Initial subgraph: {subgraph.number_of_nodes():,} nodes")

# Expand hop by hop until we hit 5,000
hop = 1
current_ego = set(top50)

while subgraph.number_of_nodes() < 5000 and hop <= 4:
    print(f"Expanding to hop {hop + 1}...")
    new_nodes = set()
    for node in current_ego:
        new_nodes.update(G.predecessors(node))
        new_nodes.update(G.successors(node))
    current_ego.update(new_nodes)
    subgraph = G.subgraph(current_ego).copy()
    print(f"  Nodes after hop {hop + 1}: {subgraph.number_of_nodes():,}")
    hop += 1

print(f"\nFinal subgraph: {subgraph.number_of_nodes():,} nodes, {subgraph.number_of
```

Initial subgraph: 567 nodes

Expanding to hop 2...

Nodes after hop 2: 567

Expanding to hop 3...

Nodes after hop 3: 4,061

Expanding to hop 4...

Nodes after hop 4: 12,569

Final subgraph: 12,569 nodes, 18,878 edges

Fix class labels on subgraph nodes

The node attributes still carry the raw '1' and '2' string labels. I'll convert them to readable names before computing metrics or visualizing.

```
In [19]: for node in subgraph.nodes():
        cls = subgraph.nodes[node].get('class', 'unknown')
```

```

if cls == '1':
    subgraph.nodes[node]['class'] = 'illicit'
elif cls == '2':
    subgraph.nodes[node]['class'] = 'licit'

unique_classes = set(nx.get_node_attributes(subgraph, 'class').values())
print(f"Class labels in subgraph: {unique_classes}")

```

Class labels in subgraph: {'unknown', 'licit', 'illicit'}

Compute centrality metrics for every node in the subgraph

Four metrics together give a fuller picture of each node's role than any single one would:

- **In-degree:** how many transactions feed into this one
- **Out-degree:** how many transactions this one feeds into
- **Betweenness:** how often this node bridges other nodes
- **PageRank:** a recursive importance score - nodes fed by important nodes score higher

```

In [20]: print("Computing centrality metrics...")

in_degree = dict(subgraph.in_degree())
out_degree = dict(subgraph.out_degree())
betweenness = nx.betweenness_centrality(subgraph, normalized=True)
pagerank = nx.pagerank(subgraph, alpha=0.85)

centrality_df = pd.DataFrame({
    'txId' : list(subgraph.nodes()),
    'in_degree' : [in_degree[n] for n in subgraph.nodes()],
    'out_degree' : [out_degree[n] for n in subgraph.nodes()],
    'betweenness' : [betweenness[n] for n in subgraph.nodes()],
    'pagerank' : [pagerank[n] for n in subgraph.nodes()],
})

centrality_df['class'] = centrality_df['txId'].map(
    nx.get_node_attributes(subgraph, 'class')
)
centrality_df['class'] = centrality_df['class'].replace({'1': 'illicit', '2': 'lici

print(f"Centrality dataframe: {centrality_df.shape}")
print(centrality_df.sort_values('betweenness', ascending=False).head(10))

```

Computing centrality metrics...

Centrality dataframe: (12569, 6)

	txId	in_degree	out_degree	betweenness	pagerank	class
6430	12665008	84	9	0.000012	0.001844	licit
8515	7951587	2	14	0.000011	0.000461	licit
3634	19670253	91	8	0.000010	0.002316	licit
1027	38538087	80	5	0.000010	0.002295	licit
3511	96936902	20	1	0.000009	0.000496	licit
1972	14193907	1	15	0.000008	0.000065	licit
3589	35202200	77	5	0.000008	0.002269	licit
5975	12663816	1	13	0.000008	0.000056	licit
3775	12691111	1	12	0.000008	0.000043	licit
11504	12678407	1	2	0.000008	0.000044	licit

Identify the top 10 most central nodes

```
In [21]: top10_nodes = centrality_df.nlargest(10, 'betweenness')['txId'].tolist()

print("Top 10 nodes by betweenness centrality:")
for i, node in enumerate(top10_nodes, 1):
    cls = nx.get_node_attributes(subgraph, 'class').get(node, 'unknown')
    print(f" {i:>2}. {node} | {cls} | betweenness: {betweenness[node]:.6f}")
```

Top 10 nodes by betweenness centrality:

1.	12665008		licit		betweenness: 0.000012
2.	7951587		licit		betweenness: 0.000011
3.	19670253		licit		betweenness: 0.000010
4.	38538087		licit		betweenness: 0.000010
5.	96936902		licit		betweenness: 0.000009
6.	14193907		licit		betweenness: 0.000008
7.	35202200		licit		betweenness: 0.000008
8.	12663816		licit		betweenness: 0.000008
9.	12691111		licit		betweenness: 0.000008
10.	12678407		licit		betweenness: 0.000008

Visualize the subgraph

Color scheme:

- **Gold** - top 10 most central nodes (override all other colors)
- **Red** - illicit transactions
- **Green** - licit transactions
- **Grey** - unknown/unlabeled transactions

I'm using spring layout with a low **k** value to keep related nodes close together without the whole graph turning into a blob.

```
In [22]: color_map = []
node_size = []
```

```

for node in subgraph.nodes():
    cls = subgraph.nodes[node].get('class', 'unknown')
    if node in top10_nodes:
        color_map.append('gold')
        node_size.append(120)
    elif cls == 'illicit':
        color_map.append('#e05c5c')
        node_size.append(40)
    elif cls == 'licit':
        color_map.append('#5ca05c')
        node_size.append(20)
    else:
        color_map.append('aaaaaa')
        node_size.append(15)

fig, ax = plt.subplots(figsize=(18, 14))
pos = nx.spring_layout(subgraph, k=0.15, seed=42)

nx.draw_networkx_nodes(subgraph, pos, node_color=color_map,
                        node_size=node_size, alpha=0.85, ax=ax)
nx.draw_networkx_edges(subgraph, pos, edge_color='cccccc',
                        arrows=True, arrowsize=6, width=0.4,
                        alpha=0.5, ax=ax)

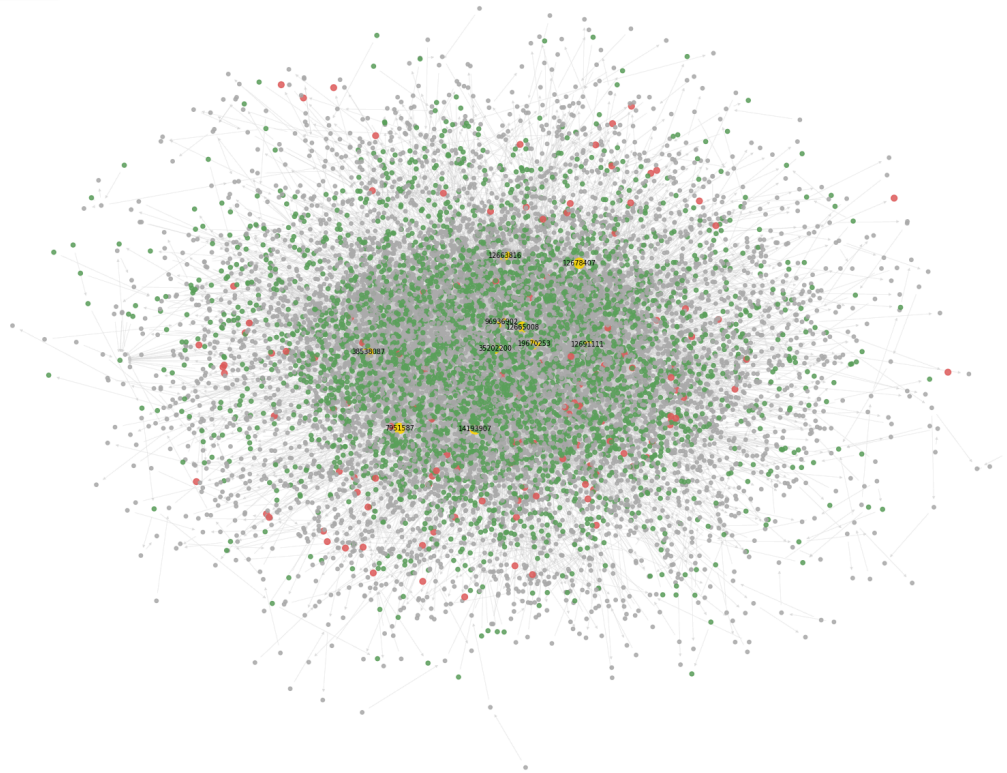
# Only Label the top 10 - Labeling every node makes it unreadable
top10_labels = {n: str(n) for n in top10_nodes}
nx.draw_networkx_labels(subgraph, pos, labels=top10_labels,
                        font_size=7, font_color='black', ax=ax)

legend_elements = [
    mpatches.Patch(color='gold', label='Top 10 central nodes'),
    mpatches.Patch(color='#e05c5c', label='Illicit'),
    mpatches.Patch(color='#5ca05c', label='Licit'),
    mpatches.Patch(color='aaaaaa', label='Unknown'),
]
ax.legend(handles=legend_elements, loc='upper left', fontsize=11)
ax.set_title('Bitcoin Transaction Subgraph – Top 50 Ego Network', fontsize=14)
ax.axis('off')

plt.tight_layout()
plt.savefig('subgraph_visualisation.png', dpi=150, bbox_inches='tight')
plt.show()
print("Saved: subgraph_visualisation.png")

```

Bitcoin Transaction Subgraph — Top 50 Ego Network



Saved: subgraph_visualisation.png

Task 3: Fraud Pattern Detection

I'm looking for two specific behavioral patterns that are common in cryptocurrency money laundering:

1. **Layering** - a transaction collects funds from multiple sources and quickly redistributes them onward. This is how launderers obscure where money came from.
2. **Fan-out** - a transaction sends to a large number of distinct destinations in a short window. This is known as structuring or smurfing — breaking one large amount into many small ones to avoid detection thresholds.

```
In [23]: timestep = nx.get_node_attributes(subgraph, 'time_step')
class_attr = nx.get_node_attributes(subgraph, 'class')

print(f"Nodes with time step data: {sum(1 for v in timestep.values() if v != -1):,}")
```

Nodes with time step data: 12,569

Detect layering behavior

A node is flagged for layering if it receives from 2 or more sources AND sends to at least 1 destination. The in-degree threshold of 2 is grounded in the actual data, checking the distribution showed 1,621 nodes meet this criteria, so it's not an arbitrary number.

```
In [24]: # Quick look at the in-degree distribution before setting thresholds
in_degs = [d for n, d in subgraph.in_degree()]
print(f"Max in-degree           : {max(in_degs)}")
print(f"Nodes with in-degree >= 2 : {sum(1 for d in in_degs if d >= 2)}")
print(f"Nodes with in-degree >= 1 : {sum(1 for d in in_degs if d >= 1)}")
```

```
Max in-degree           : 284
Nodes with in-degree >= 2 : 1621
Nodes with in-degree >= 1 : 6710
```

```
In [25]: print("Detecting layering behavior...")

layering_nodes = []

for node in subgraph.nodes():
    sources      = list(subgraph.predecessors(node))
    destinations = list(subgraph.successors(node))
    in_deg       = len(sources)
    out_deg      = len(destinations)

    # Must receive from at least 2 sources and send to at least 1 destination
    if in_deg < 2 or out_deg < 1:
        continue

    # Gather time steps from this node and its neighbors
    neighbour_ts = set()
    node_ts = timestep.get(node, -1)
    if node_ts != -1:
        neighbour_ts.add(node_ts)

    for n in sources + destinations:
        ts = timestep.get(n, -1)
        if ts != -1:
            neighbour_ts.add(ts)

    sorted_ts = sorted(neighbour_ts) if neighbour_ts else [-1]
    ts_range  = sorted_ts[-1] - sorted_ts[0] if len(sorted_ts) > 1 else 0
    num_ts    = len(sorted_ts)

    layering_nodes.append({
        'txId'       : node,
        'in_degree'   : in_deg,
        'out_degree'  : out_deg,
        'time_steps'  : sorted_ts,
        'num_time_steps': num_ts,
        'ts_range'    : ts_range,
        'class'       : class_attr.get(node, 'unknown')
    })
```

```
layering_df = pd.DataFrame(layering_nodes)
layering_df['class'] = layering_df['class'].replace({'1': 'illicit', '2': 'licit'})
print(f"Layering nodes flagged: {len(layering_df)}")
```

Detecting layering behavior...
 Layering nodes flagged: 993

Detect fan-out behavior

Rather than picking an arbitrary number for the fan-out threshold I'm using the 95th percentile of out-degree. That way the threshold is determined by the data itself, nodes in the top 5% of outgoing connections within a tight time window get flagged.

```
In [26]: print("Detecting fan-out behavior...")

out_degrees      = dict(subgraph.out_degree())
fanout_threshold = np.percentile(list(out_degrees.values()), 95)
print(f"Fan-out threshold (95th percentile): {fanout_threshold:.1f}")

fanout_nodes = []

for node in subgraph.nodes():
    destinations = list(subgraph.successors(node))
    out_deg      = len(destinations)

    if out_deg < fanout_threshold:
        continue

    dest_ts = set()
    for dest in destinations:
        ts = timestep.get(dest, -1)
        if ts != -1:
            dest_ts.add(ts)

    if len(dest_ts) == 0:
        continue

    sorted_ts = sorted(dest_ts)
    ts_range  = sorted_ts[-1] - sorted_ts[0]

    # Only flag if destinations are reached within 2 time steps
    # Spreading over a long period is less suspicious
    if ts_range <= 2:
        fanout_nodes.append({
            'txId'           : node,
            'out_degree'     : out_deg,
            'unique_destinations': len(set(destinations)),
            'time_steps'     : sorted_ts,
            'ts_range'       : ts_range,
            'class'          : class_attr.get(node, 'unknown')
        })

fanout_df = pd.DataFrame(fanout_nodes)
```

```
fanout_df['class'] = fanout_df['class'].replace({'1': 'illicit', '2': 'licit'})  
print(f"Fan-out nodes flagged: {len(fanout_df)}")
```

Detecting fan-out behavior...

Fan-out threshold (95th percentile): 5.0

Fan-out nodes flagged: 638

Cross-reference findings with class labels

```
In [27]: print("LAYERING - label breakdown:")  
         print(layering_df['class'].value_counts())  
  
         print("\nTop 10 layering nodes by in-degree:")  
         print(layering_df.sort_values('in_degree', ascending=False).head(10)[  
             ['txId', 'in_degree', 'out_degree', 'num_time_steps', 'class']  
         ]).to_string(index=False))  
  
         print("\nFAN-OUT - label breakdown:")  
         print(fanout_df['class'].value_counts())  
  
         print("\nTop 10 fan-out nodes by out-degree:")  
         print(fanout_df.sort_values('out_degree', ascending=False).head(10)[  
             ['txId', 'out_degree', 'unique_destinations', 'ts_range', 'class']  
         ]).to_string(index=False))
```

LAYERING - label breakdown:

```
class
unknown    597
licit       394
illicit      2
Name: count, dtype: int64
```

Top 10 layering nodes by in-degree:

txId	in_degree	out_degree	num_time_steps	class
70245040	133	1	1	unknown
12780176	109	1	1	licit
73330801	104	1	1	licit
6418	97	2	1	licit
156313815	95	1	1	licit
156063345	94	4	1	licit
73227485	94	2	1	licit
47066538	93	4	1	licit
35617620	92	1	1	licit
19670253	91	8	1	licit

FAN-OUT - label breakdown:

```
class
unknown    488
licit       150
Name: count, dtype: int64
```

Top 10 fan-out nodes by out-degree:

txId	out_degree	unique_destinations	ts_range	class
2984918	472	472	0	licit
102570	122	122	0	unknown
7952	99	99	0	licit
143705	92	92	0	unknown
565334	90	90	0	licit
4600600	76	76	0	unknown
792948	74	74	0	unknown
36460504	65	65	0	unknown
65253	53	53	0	unknown
2347070	32	32	0	unknown

Flag suspicious unknown nodes

An unknown node can't be confirmed illicit by label alone. But if it's flagged for a suspicious pattern AND sits directly next to a confirmed illicit node, that's a compliance risk regardless of the missing label. These are the nodes that most warrant manual investigation.

```
In [28]: all_flagged = set()
if len(layering_df) > 0:
    all_flagged.update(layering_df['txId'].tolist())
if len(fanout_df) > 0:
    all_flagged.update(fanout_df['txId'].tolist())

suspicious_unknowns = []
```

```

for node in all_flagged:
    if class_attr.get(node, 'unknown') != 'unknown':
        continue

    neighbours = list(subgraph.predecessors(node)) + list(subgraph.successors(node))
    illicit_neighbours = [
        n for n in neighbours
        if class_attr.get(n, 'unknown') == 'illicit'
    ]

    if len(illicit_neighbours) > 0:
        suspicious_unknowns.append({
            'txId' : node,
            'illicit_neighbours' : len(illicit_neighbours),
            'in_degree' : subgraph.in_degree(node),
            'out_degree' : subgraph.out_degree(node),
            'pagerank' : pagerank.get(node, 0),
        })

suspicious_unknowns_df = pd.DataFrame(suspicious_unknowns)

if len(suspicious_unknowns_df) > 0:
    print(f"Unknown nodes flagged by pattern AND connected to illicit nodes: {len(suspicious_unknowns_df)}")
    print(suspicious_unknowns_df.sort_values('illicit_neighbours', ascending=False))
else:
    print("No unknown nodes with direct illicit connections found among flagged nodes")

```

Unknown nodes flagged by pattern AND connected to illicit nodes: 20

txId	illicit_neighbours	in_degree	out_degree	pagerank
12716727	5	1	11	0.000082
12665098	4	0	8	0.000037
84997416	3	3	1	0.000131
12776716	3	1	14	0.000068
12777206	2	0	8	0.000037
55801561	2	2	6	0.000053
55735705	2	0	5	0.000037
2060002	2	1	9	0.000068
5866881	2	3	11	0.000100
41428160	2	0	9	0.000037

Fraud pattern summary

```

In [29]: total_flagged = len(all_flagged)
pct = (total_flagged / subgraph.number_of_nodes()) * 100

print(f"Total subgraph nodes : {subgraph.number_of_nodes():,}")
print(f"Layering nodes : {len(layering_df):,}")
print(f"Fan-out nodes : {len(fanout_df):,}")
print(f"Suspicious unknowns : {len(suspicious_unknowns_df):,}")
print(f"Total unique flagged : {total_flagged:,}")
print(f"As % of subgraph : {pct:.2f}%")

```

Total subgraph nodes : 12,569
 Layering nodes : 993
 Fan-out nodes : 638
 Suspicious unknowns : 20
 Total unique flagged : 1,391
 As % of subgraph : 11.07%

Task 4: Business Report

See the separate `compliance_report` file submitted with this notebook.

Task 5: Interactive HTML Dashboard

Building a single-page self-contained HTML dashboard that answers: **who are the riskiest nodes and how are they connected?**

The composite risk score is defined as:

$$\text{risk} = 0.30 \times \text{betweenness} + 0.25 \times \text{pagerank} + 0.25 \times \text{out_degree} + 0.20 \times \text{illicit_flag}$$

All metrics are normalized to 0-1 before combining so no single metric dominates. The illicit flag gives a 20% boost to confirmed fraudulent nodes.

```
In [30]: import json

def normalise(d):
    vals = np.array(list(d.values()), dtype=float)
    mn, mx = vals.min(), vals.max()
    if mx == mn:
        return {k: 0.0 for k in d}
    return {k: (v - mn) / (mx - mn) for k, v in d.items()}

print("Computing risk scores...")
bc_score = normalise(nx.betweenness centrality(subgraph, normalized=True))
pr_score = normalise(nx.pagerank(subgraph, alpha=0.85))
out_score = normalise(dict(subgraph.out_degree()))

rows = []
for node in subgraph.nodes():
    cls = subgraph.nodes[node].get('class', 'unknown')
    label_flag = 1.0 if cls == 'illicit' else 0.0
    risk = (
        0.30 * bc_score.get(node, 0) +
        0.25 * pr_score.get(node, 0) +
        0.25 * out_score.get(node, 0) +
        0.20 * label_flag
    )
```

```

rows.append({
    'txId'      : str(node),
    'class'     : cls,
    'betweenness': round(bc_score.get(node, 0), 6),
    'pagerank'  : round(pr_score.get(node, 0), 6),
    'out_degree' : subgraph.out_degree(node),
    'in_degree'  : subgraph.in_degree(node),
    'risk_score' : round(risk, 4),
    'time_step'  : subgraph.nodes[node].get('time_step', -1),
})

risk_df = pd.DataFrame(rows).sort_values('risk_score', ascending=False)
risk_df['class'] = risk_df['class'].replace({'1': 'illicit', '2': 'licit'})
top20 = risk_df.head(20)
top10_ids = risk_df.head(10)['txId'].tolist()

print(f"Risk scores computed for {len(risk_df):,} nodes")
print(f"Top risk score: {risk_df['risk_score'].max():.4f}")
print(risk_df['class'].value_counts())

```

Computing risk scores...

Risk scores computed for 12,569 nodes

Top risk score: 0.3609

class

unknown 8448

licit 3994

illicit 127

Name: count, dtype: int64

```

In [31]: print("Preparing network layout...")

top500_ids = set(risk_df.head(500)['txId'].astype(str).tolist())
viz_nodes = [n for n in subgraph.nodes() if str(n) in top500_ids]
viz_subgraph = subgraph.subgraph(viz_nodes)

pos = nx.spring_layout(viz_subgraph, k=0.3, seed=42)

edge_x, edge_y = [], []
for u, v in viz_subgraph.edges():
    x0, y0 = pos[u]
    x1, y1 = pos[v]
    edge_x += [x0, x1, None]
    edge_y += [y0, y1, None]

node_traces = {'x': [], 'y': [], 'text': [], 'size': []}
for cls in ['illicit', 'licit', 'unknown', 'top10']:

for node in viz_subgraph.nodes():
    x, y = pos[node]
    cls = viz_subgraph.nodes[node].get('class', 'unknown')
    nid = str(node)
    rs = risk_df[risk_df['txId'] == nid]['risk_score'].values
    score = rs[0] if len(rs) > 0 else 0
    label = f"ID: {nid}<br>Class: {cls}<br>Risk Score: {score:.4f}"
    bucket = 'top10' if nid in top10_ids else cls
    if bucket not in node_traces:

```

```

        bucket = 'unknown'
        node_traces[bucket]['x'].append(x)
        node_traces[bucket]['y'].append(y)
        node_traces[bucket]['text'].append(label)
        node_traces[bucket]['size'].append(18 if nid in top10_ids else 7)

ts_df      = risk_df[risk_df['time_step'] > 0].copy()
ts_grouped = ts_df.groupby(['time_step', 'class']).size().reset_index(name='count')

all_nodes_data = risk_df.to_dict(orient='records')
top20_data     = top20.to_dict(orient='records')
ts_data        = ts_grouped.to_dict(orient='records')
edge_trace_data = {'x': edge_x, 'y': edge_y}

print("Layout ready.")

```

Preparing network layout...

Layout ready.

```

In [32]: html_content = f"""<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Bitcoin Fraud Network Dashboard</title>
<script src="https://cdn.plot.ly/plotly-latest.min.js"></script>
<style>
  * {{ margin: 0; padding: 0; box-sizing: border-box; }}
  body {{ font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif;
    background: #0f1117; color: #e0e0e0; }}
  .header {{ background: #1a1d27; padding: 20px 30px; border-bottom: 1px solid #2a2
    display: flex; justify-content: space-between; align-items: center; }}
  .header h1 {{ font-size: 20px; font-weight: 600; color: #ffffff; }}
  .header p {{ font-size: 12px; color: #888; margin-top: 4px; }}
  .badge {{ background: #e05c5c; color: white; padding: 4px 10px;
    border-radius: 12px; font-size: 12px; font-weight: 500; }}
  .controls {{ background: #1a1d27; padding: 12px 30px;
    border-bottom: 1px solid #2a2d3a; display: flex;
    align-items: center; gap: 12px; }}
  .controls label {{ font-size: 13px; color: #aaa; }}
  .controls select {{ background: #2a2d3a; color: #e0e0e0; border: 1px solid #3a3d4
    padding: 6px 12px; border-radius: 6px; font-size: 13px; cursor
  .grid {{ display: grid; grid-template-columns: 1fr 1fr;
    grid-template-rows: auto auto; gap: 16px; padding: 16px; }}
  .card {{ background: #1a1d27; border-radius: 10px; border: 1px solid #2a2d3a; pad
  .card.full {{ grid-column: 1 / -1; }}
  .card h2 {{ font-size: 14px; font-weight: 600; color: #ccc; margin-bottom: 12px;
    text-transform: uppercase; letter-spacing: 0.5px; }}
  .chart {{ width: 100%; height: 420px; }}
  table {{ width: 100%; border-collapse: collapse; font-size: 13px; }}
  thead tr {{ background: #2a2d3a; }}
  th {{ padding: 10px 12px; text-align: left; color: #aaa; font-weight: 500;
    font-size: 12px; text-transform: uppercase; }}
  td {{ padding: 9px 12px; border-bottom: 1px solid #2a2d3a; color: #ddd; }}
  tr:hover td {{ background: #1f2235; }}
  .pill {{ display: inline-block; padding: 2px 8px; border-radius: 10px;

```

```

        font-size: 11px; font-weight: 500; }}
.pill.illicit {{ background: #3d1f1f; color: #e05c5c; }}
.pill.licit   {{ background: #1f3d1f; color: #5ca05c; }}
.pill.unknown {{ background: #2a2d3a; color: #aaaaaa; }}
.pill.top10   {{ background: #3d3500; color: #FFD700; }}
.stat-row {{ display: flex; gap: 16px; margin-bottom: 16px; }}
.stat {{ background: #2a2d3a; border-radius: 8px; padding: 12px 16px; flex: 1; }}
.stat.val {{ font-size: 22px; font-weight: 700; color: #fff; }}
.stat.lbl {{ font-size: 11px; color: #888; margin-top: 2px; }}
</style>
</head>
<body>
<div class="header">
  <div>
    <h1>Bitcoin Fraud Network - Risk Dashboard</h1>
    <p>Elliptic Dataset · Subgraph Analysis · {len(risk_df):,} nodes · {subgraph.nu
  </div>
  <div class="badge">Compliance Review</div>
</div>
<div class="controls">
  <label>Filter by class:</label>
  <select id="classFilter" onchange="applyFilter()">
    <option value="all">All Classes</option>
    <option value="illicit">Illicit</option>
    <option value="licit">Licit</option>
    <option value="unknown">Unknown</option>
  </select>
  <label style="margin-left:20px; color:#666; font-size:12px;">Top 500 nodes in gra
</div>
<div style="padding: 16px 16px 0;">
  <div class="stat-row">
    <div class="stat"><div class="val">{len(risk_df):,}</div><div class="lbl">Total
    <div class="stat"><div class="val" style="color:#e05c5c">{(risk_df['class']=='i
    <div class="stat"><div class="val" style="color:#5ca05c">{(risk_df['class']=='l
    <div class="stat"><div class="val" style="color:#aaa">{(risk_df['class']=='unkn
    <div class="stat"><div class="val" style="color:#FFD700">{len(layering_df):,}</
    <div class="stat"><div class="val" style="color:#FFD700">{len(fanout_df):,}</di
  </div>
</div>
<div class="grid">
  <div class="card"><h2>Network Graph – Top 500 Nodes by Risk</h2><div id="networkC
  <div class="card"><h2>Transaction Activity by Time Step</h2><div id="timeChart" c
  <div class="card full"><h2>Top 20 Nodes by Composite Risk Score</h2><div id="risk
</div>
<script>
const ALL_NODES = {json.dumps(all_nodes_data)};
const TOP20     = {json.dumps(top20_data)};
const TS_DATA   = {json.dumps(ts_data)};
const EDGE_TRACE = {json.dumps(edge_trace_data)};
const NODE_TRACE = {json.dumps(node_traces)};
const TOP10_IDS = {json.dumps(top10_ids)};
const COLOR = {{ illicit:'#e05c5c', licit:'#5ca05c', unknown:'#aaaaaa', top10:'#FFD

function buildNetwork(filterClass) {{
  const edgeTrace = {{ x: EDGE_TRACE.x, y: EDGE_TRACE.y, mode:'lines',
    line:{{ width:0.4, color:'#333' }}, hoverinfo:'none', type:'scatter' }};

```

```

const traces = [edgeTrace];
const labels = ['illicit','licit','unknown','top10'];
const names = ['Illicit','Licit','Unknown','Top 10 Risk'];
labels.forEach((cls, i) => {{
  const d = NODE_TRACE[cls];
  if (!d || d.x.length === 0) return;
  let xs=d.x, ys=d.y, ts=d.text, sz=d.size;
  if (filterClass !== 'all') {{
    const keep = ts.map(t => t.includes('Class: '+filterClass) ||
      (filterClass==='illicit' && TOP10_IDS.includes(t.match(/ID: (\\S+)/)?.[1]))
    xs=xs.filter((_,i)=>keep[i]); ys=ys.filter((_,i)=>keep[i]);
    ts=ts.filter((_,i)=>keep[i]); sz=sz.filter((_,i)=>keep[i]);
  }}
  traces.push({{ x:xs, y:ys, text:ts, mode:'markers',
    marker:{{ size:sz, color:COLOR[cls], opacity:0.85,
      line:{{ width:cls==='top10'?1.5:0, color:'#fff' }} }},
    name:names[i], hovertemplate:'%{{text}}<extra></extra>', type:'scatter' }}));
}});
Plotly.newPlot('networkChart', traces, {{
  paper_bgcolor:'transparent', plot_bgcolor:'transparent', showlegend:true,
  legend:{{ font:{{ color:'#aaa', size:11 }}, bgcolor:'rgba(0,0,0,0)' }},
  xaxis:{{ showgrid:false, zeroline:false, showticklabels:false }},
  yaxis:{{ showgrid:false, zeroline:false, showticklabels:false }},
  margin:{{ l:10, r:10, t:10, b:10 }}, hovermode:'closest'
}}, {{responsive:true}});
}}

function buildTimeSeries(filterClass) {{
  const classes = filterClass==='all' ? ['illicit','licit','unknown'] : [filterClass];
  const traces = classes.map(cls => {{
    const filtered = TS_DATA.filter(d => d.class===cls);
    filtered.sort((a,b) => a.time_step-b.time_step);
    return {{ x:filtered.map(d=>d.time_step), y:filtered.map(d=>d.count),
      name:cls.charAt(0).toUpperCase()+cls.slice(1), type:'scatter', mode:'lines+markers',
      line:{{ color:COLOR[cls], width:2 }}, marker:{{ size:4, color:COLOR[cls] }} }
  }});
  Plotly.newPlot('timeChart', traces, {{
    paper_bgcolor:'transparent', plot_bgcolor:'transparent',
    xaxis:{{ title:'Time Step', color:'#888', gridcolor:'#2a2d3a', tickfont:{{ color:'black', size:12 }},
    yaxis:{{ title:'Transaction Count', color:'#888', gridcolor:'#2a2d3a', tickfont:{{ color:'black', size:12 }},
    legend:{{ font:{{ color:'#aaa', size:11 }}, bgcolor:'rgba(0,0,0,0)' }},
    margin:{{ l:50, r:20, t:10, b:50 }}, hovermode:'x unified'
  }}, {{responsive:true}});
}}

function buildTable(filterClass) {{
  const data = filterClass==='all' ? TOP20 : ALL_NODES.filter(d=>d.class===filterClass);
  let html = `<table><thead><tr><th>#</th><th>Transaction ID</th><th>Class</th><th>Risk Score</th><th>Betweenness</th><th>PageRank</th><th>Out Degree</th><th>In Degree</th></tr></thead><tbody>`;
  data.forEach((row, i) => {{
    const isTop10 = TOP10_IDS.includes(row.txId);
    const pillCls = isTop10 ? 'top10' : row.class;
    const pillLbl = isTop10 ? '★ Top 10' : row.class;
    html += `<tr><td style="color:#666">${{i+1}}</td><td style="font-family:monospace;font-size:12px">${{row.txId}}</td><td style="font-family:monospace;font-size:12px">${{pillCls}}</td><td style="font-family:monospace;font-size:12px">${{pillLbl}}</td><td style="font-family:monospace;font-size:12px">${{row.riskScore}}</td><td style="font-family:monospace;font-size:12px">${{row.betweenness}}</td><td style="font-family:monospace;font-size:12px">${{row.pagerank}}</td><td style="font-family:monospace;font-size:12px">${{row.outDegree}}</td><td style="font-family:monospace;font-size:12px">${{row.inDegree}}</td></tr>`;
  }});
  return html;
}

```

```

        <td><span class="pill ${pillCls}">${pillLbl}</span></td>
        <td style="color:#FFD700;font-weight:600">${row.risk_score}</td>
        <td>${row.betweenness}</td><td>${row.pagerank}</td>
        <td>${row.out_degree}</td><td>${row.in_degree}</td></tr>`;
    });
    html += '</tbody></table>';
    document.getElementById('riskTable').innerHTML = html;
  }}

function applyFilter() {{
  const cls = document.getElementById('classFilter').value;
  buildNetwork(cls); buildTimeSeries(cls); buildTable(cls);
}}

buildNetwork('all'); buildTimeSeries('all'); buildTable('all');
</script>
</body>
</html>""

with open('elliptic_fraud_dashboard.html', 'w', encoding='utf-8') as f:
    f.write(html_content)

print("Dashboard saved: elliptic_fraud_dashboard.html")

```

Dashboard saved: elliptic_fraud_dashboard.html

In []:

In []: